

BrandBridge AI

Mobile responsiveness pass, critical bug fix,
and six platform additions

Date	19 August 2026
Repository	E:\brandbridgeai
Branch	main — working tree, not committed
Files touched	48 modified · 14 created
Verification	TypeScript clean · 58/58 tests pass · build OK

Nothing in this change set has been committed or pushed. The dev server was not started.

1 Executive summary

Work happened in two passes. The first was the mobile responsiveness sweep you asked for, which surfaced a genuine rendering bug along the way. The second implemented the six gaps identified during the audit, in the priority order agreed.

17

Broken template literals repaired in the brand profile form

62

Individual responsive corrections across modals, grids and layout

9

AI endpoints now rate limited against abuse and cost overrun

58

Automated tests added, all passing, from a zero-test baseline

What changed, by category

Type	Change	Impact
BUG	Brand profile form rendered escaped template literals as literal CSS class text	Page was partly non-functional
MOBILE	iOS zoom-on-focus, viewport units, hover-only controls, cramped grids, modal padding	Phone usability
FEATURE	Error, loading and 404 boundaries across the app	No more blank screens
FEATURE	Live message polling in chat and conversation list	Messaging now usable
HARDENING	Per-user rate limiting on all Gemini-backed routes	Cost & abuse control
FEATURE	Lazy image loading, sitemap, robots, PWA manifest, test suite	Performance, SEO, safety net

How to read this document

Sections 2–4 explain what changed and why. **Section 5 is the verification guide** — it tells you exactly which page to open, at which screen width, and what you should see. Section 7 lists the limits of what was done.

2 Critical bug found and fixed

Escaped template literals in the brand profile form

src/components/dashboard/brand/BrandProfileForm.tsx

Seventeen JSX template literals in this file had their interpolation escaped — `\${}` instead of `${}`. JavaScript therefore treated the entire expression as literal text rather than evaluating it, and that raw text was passed straight into the `className` and `style` attributes.

```
// Before – the backslash disables interpolation entirely
className={`... transition-all duration-300 \${activeTab === 'basic' ? 'block' : 'hidden sm:hidden'}`}

// After
className={`... transition-all duration-300 ${activeTab === 'basic' ? 'block' : 'hidden sm:hidden'}`}
```

The practical effect was that several interactive parts of the brand profile page did nothing at all:

Broken behaviour	Symptom before the fix
Mobile accordion sections	Sections never expanded or collapsed on tap — the show/hide class was never computed
Desktop tab highlighting	Active tab never received its underline or accent colour
Profile completeness bar	Width was the literal string, so the bar never reflected the real percentage
Registered-business toggle	Switch never moved or changed colour when toggled
Availability status dot	Colour never matched the selected status
Floating save bar	Never slid into view when the form became dirty

Secondary fix in the same file

The floating save bar offset the sidebar with `md:pl-[17rem]`, but the sidebar only becomes a static column at the `lg` breakpoint. Between 768 px and 1024 px the bar was indented for a sidebar that was not there. Changed to `lg:pl-[17rem]`.

Why this matters most

This was not a styling preference — it was broken code that TypeScript cannot catch, because an escaped template literal is still a perfectly valid string. It would not have surfaced in a build or a type check.

3 Mobile responsiveness

The application already had a solid responsive foundation — the sidebar drawer, the two-pane messages flow, and horizontally scrolling tab strips were all in place. The work below closes the gaps that remained.

3.1 iOS auto-zoom on form focus

[src/styles/brandbridge.css](#)

Safari on iOS zooms the whole page whenever a focused input has a font size below 16 px. Every field in the app used `0.875rem` (14 px), so tapping any input jumped the layout and the user had to pinch back out. Form controls are now pinned to 16 px on screens up to 640 px.

```
/* iOS Safari zooms the viewport when a focused field is under 16px.
   Keep every form control at 16px on phones so focus never zooms. */
@media (max-width: 640px) {
  .bb-input, input.bb-input, select.bb-input, textarea.bb-input,
  input:not([type="checkbox"]):not([type="radio"]):not([type="range"]),
  select, textarea { font-size: 16px; }
}
```

3.2 Viewport height units in modals

Sixteen components sized themselves with `vh`. On mobile browsers `vh` is measured against the viewport *including* the collapsible address bar, so modal content and action buttons were pushed below the visible area. All were converted to `dvh`, which tracks the actual visible height.

3.3 Hover-only controls unreachable on touch

[src/components/messages/ChatWindow.tsx](#)

The edit and unsend buttons on your own messages were `opacity-0 group-hover:opacity-100`. Touch devices have no hover state, so on a phone these controls could never be revealed — the features were effectively absent on mobile. They are now visible by default and only hover-gated from `sm` upward, preserving the clean desktop appearance.

3.4 Everything else

Fix	Detail	Files
Messages panel height	Was a single fixed <code>100vh-80px</code> at every breakpoint; now uses <code>dvh</code> and matches the real header plus per-breakpoint page padding	1
Notification dropdown	Fixed <code>w-80</code> (320 px) overflowed the screen on small phones; now <code>w-[calc(100vw-1.5rem)] max-w-80</code>	1
Two-column form grids	Date pickers and paired selects were squeezed side by side on phones; now stack below <code>sm</code>	3
Three-column stat and button grids	Labels such as “Hire Success Rate” wrapped to three lines in ~100 px columns; now stack below <code>sm</code>	2
Modal padding	<code>p-6</code> consumed 48 px of a 360 px screen; now <code>p-4 sm:p-6</code> across header, body and footer	5
Auth card padding	<code>p-8</code> reduced to <code>p-6</code> below <code>sm</code>	1

Fix	Detail	Files
Viewport configuration	Explicit <code>viewport</code> export added with <code>viewportFit: "cover"</code> for notched devices, plus light and dark theme colours	1

Page zoom was deliberately left enabled

A common way to "fix" iOS zoom is `maximum-scale=1` , which also blocks the user from pinch-zooming. That breaks accessibility for anyone with low vision, so the font-size approach in 3.1 was used instead.

4 Features added

4.1 Error, loading and 404 boundaries NEW

The application previously had no `error.tsx`, `loading.tsx` or `not-found.tsx` anywhere. A failed API call or a thrown render produced either a blank white screen or an unhandled React crash, with no way back other than the browser's back button. Six files now cover every failure path:

File	Catches
<code>src/app/error.tsx</code>	Any uncaught render or data error, with a retry button and a route back to the dashboard
<code>src/app/global-error.tsx</code>	Failures in the root layout itself. Renders its own <code><html></code> shell with inline styles, since neither the theme provider nor Tailwind is available at that point
<code>src/app/not-found.tsx</code>	Unmatched routes — a branded 404 rather than the Next.js default
<code>src/app/loading.tsx</code>	Top-level route transitions
<code>src/app/(protected)/loading.tsx</code>	Dashboard navigation, using a skeleton shaped like the real dashboard (header, stat row, content cards) so the layout does not jump when data lands
<code>src/app/(protected)/error.tsx</code>	Dashboard section failures. Renders <i>inside</i> the shell, so the sidebar and header stay usable and the user is not ejected from the app

Where Next.js supplies an error `digest`, it is displayed so a user can quote it in a support request and you can match it to your server logs.

4.2 Live messaging NEW

`src/components/messages/ChatWindow.tsx` · `MessagesCenter.tsx`

Chat previously fetched messages only when you switched conversation or sent something. A message from the other participant never appeared — the recipient had to reload the page to discover it, which made the messaging feature effectively unusable for real conversation.

Messages now poll every 5 seconds and the conversation list every 15 seconds. Four details make this safe rather than merely functional:

- **No spinner flash.** Background polls run in a silent mode that never touches the loading state, so the UI does not flicker every five seconds.
- **No pointless re-renders.** Each poll is structurally compared against the current list; if nothing changed, React state is left untouched.
- **Scroll position is respected.** The view auto-scrolls only when new messages arrive *and* you are already near the bottom. Someone reading back through history is never yanked away.
- **Paused in background tabs.** Polling stops while the tab is hidden and fires an immediate catch-up fetch on return, so a backgrounded tab costs nothing.

The conversation-list refresh additionally preserves any unsent draft conversation and never steals the selection you are currently viewing.

4.3 Rate limiting on AI endpoints HARDENING

`src/lib/rate-limit.ts` – applied to all 9 routes under `src/app/api/ai/`

Every Gemini-backed route was authenticated but otherwise unlimited: any logged-in account could issue unbounded paid API calls. Two budgets now apply per user:

Budget	Limit	Purpose
Per route	12 / minute	Stops one endpoint being hammered
Across all AI routes	30 / minute	Stops the nine routes being summed to bypass the per-route cap

Exhausting either returns HTTP 429 with a `Retry-After` header and standard `X-RateLimit-*` headers, plus a plain-language message the UI can display directly.

Deployment caveat — read before going to production

Counters are held in process memory. On a single long-lived server that is exactly right: no extra infrastructure, no network hop. On Vercel or any horizontally scaled deployment, *each instance keeps its own counters*, so the real ceiling becomes limit × instances. Every call routes through a single `hit()` function specifically so this can be swapped for a Redis `INCR / EXPIRE` in one place when you scale out.

4.4 Image loading NEW

27 raw `<img>` tags across 18 files now carry `loading="lazy"` and `decoding="async"`, so off-screen avatars, logos and portfolio thumbnails no longer block initial render or compete for bandwidth on a mobile connection.

A full migration to `next/image` was deliberately not done — see Section 7.

4.5 SEO and PWA NEW

File	Provides
src/app/sitemap.ts	Lists only the four genuinely public routes. Auth-gated pages are excluded — indexing them would just feed crawlers redirects
src/app/robots.ts	Blocks <code>/api/</code> , <code>/dashboard/</code> , <code>/onboarding/</code> , <code>/verify-otp</code> and <code>/reset-password</code> , the last because it carries single-use tokens
src/app/manifest.ts	Installable PWA metadata — standalone display, brand colours, portrait orientation, <code>/dashboard</code> as start URL
src/app/layout.tsx	<code>metadataBase</code> so Open Graph and manifest URLs resolve absolutely instead of relatively

4.6 Test suite NEW

The project had no test framework at all. Vitest is now configured with the `@` path alias resolved, and 58 tests cover the pure logic most likely to break silently:

Suite	Covers
src/lib/rate-limit.test.ts	Limit enforcement, per-key isolation, window reset boundaries (including one millisecond before expiry), retry-after arithmetic, the 429 response shape, and that the global ceiling cannot be bypassed by spreading calls across routes. Uses fake timers, so it is fast and deterministic
src/lib/profile-completeness.test.ts	Brand, freelancer and generic role scoring; empty-string versus missing fields; zero hourly rate treated as unset; human-readable labels

Suite	Covers
src/lib/roles.test.ts	Role validation, dashboard path construction, and that every nav item is scoped to its own role namespace with no duplicate destinations

```
npm test          # single run
npm run test:watch # watch mode
npm run test:coverage
```


5 How to verify — step by step

The dev server has not been started

You asked to be consulted first, so nothing was launched. Start it yourself with `npm run dev` and open `http://localhost:3000`.

5.1 Checks that need no browser

```
npm test      → expect: PASS (58) FAIL (0)
npx tsc --noEmit → expect: No errors found
npm run build  → expect: exit code 0
```

5.2 Setting up a phone viewport

In Chrome press **F12**, then **Ctrl+Shift+M** to toggle device emulation. Test at **360×740** (typical Android), **390×844** (iPhone), and **768** then **1024** px to confirm the breakpoint transitions.

5.3 The critical bug fix

`/dashboard/brand/profile` – log in as a brand account

- ☐ **At 360 px:** tap each section header (Basic Info, Industry & Audience, Collaboration Prefs, Verification). Each must expand and collapse, and its chevron must rotate. Before the fix nothing happened.
- ☐ **At 1280 px:** click through the four tabs. The active one must show a purple underline and purple text.
- ☐ Check the completeness bar at the top — the filled width must visually match the percentage shown beside it.
- ☐ Open Verification & Social and toggle "registered business". The switch knob must slide across and change colour.
- ☐ Edit any field. A save bar must slide up from the bottom. At 800 px wide confirm it is *not* indented as if a sidebar were present.

5.4 Mobile responsiveness

- ☐ **iOS zoom (needs a real iPhone or iOS simulator — desktop Chrome cannot reproduce this).** Open `/login` and tap the email field. The page must not zoom in. Confirm you can still pinch-zoom manually.
- ☐ **Modals.** At 360 px open New Campaign from `/dashboard/brand/campaigns`. The Type/Goal and Start/End Date pairs must stack vertically, not sit squeezed side by side. The footer buttons must be reachable without the modal running off-screen.
- ☐ **Chat controls on touch.** Open `/dashboard/brand/messages` in device emulation with touch simulation on. Your own message bubbles must show edit and unsend icons *without* hovering. At desktop width they should stay hidden until hover.
- ☐ **Messages height.** At 360 px the message composer must sit fully on screen with no page-level vertical scrollbar. Repeat at 768 px and 1280 px.
- ☐ **Notification dropdown.** At 360 px tap the bell in the header. The panel must stay inside the screen with no horizontal scrollbar appearing.
- ☐ **Global overflow check.** On every page at 360 px, confirm no horizontal scrollbar. Paste this in the console to find any offender:

```
[...document.querySelectorAll('*')]
  .filter(el => el.getBoundingClientRect().right > document.documentElement.clientWidth + 1)
  .forEach(el => console.log(el)); // expect: nothing logged
```

5.5 Error and loading states

- ❑ **404.** Visit `/this-page-does-not-exist`. Expect the branded 404 with Dashboard and Home buttons, not the plain Next.js page.
- ❑ **Loading skeleton.** In DevTools → Network, set throttling to "Slow 4G", then navigate between dashboard sections. A skeleton shaped like the dashboard must appear instead of a blank area.
- ❑ **Error boundary.** In DevTools → Network, tick "Offline", then navigate to a dashboard section that fetches data. Expect the "This section could not be loaded" panel *with the sidebar and header still present and usable*, plus a working Retry button once you go back online.

5.6 Live messaging

This needs two accounts. Use a normal window and an incognito window side by side.

- ❑ Open the same conversation as both users. Send from A — it must appear in B's window within about 5 seconds, with no refresh.
- ❑ In B, scroll up into older messages and keep them in view. Send again from A. B must *not* be scrolled away from what they were reading.
- ❑ With B scrolled to the bottom, send from A. B must auto-scroll to reveal the new message.
- ❑ Watch B's UI for five seconds — no spinner should flash and no visible flicker should occur on each poll.
- ❑ Open DevTools → Network in B, switch to another browser tab for 30 seconds, then return. Requests must pause while hidden and one catch-up request should fire on return.
- ❑ Start a brand-new conversation from A. It must appear in B's conversation list within about 15 seconds.

5.7 AI rate limiting

Log in, open DevTools → Console on any page of the app, and paste:

```
for (let i = 1; i <= 15; i++) {  
  const r = await fetch('/api/ai/hub', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ type: 'consultant', prompt: 'test' })  
  });  
  console.log(i, r.status, r.headers.get('X-RateLimit-Remaining'));  
}
```

Expect the first 12 to return `200` with a decreasing remaining count, then `429` from the 13th onward. Wait 60 seconds and confirm requests succeed again. To confirm the global ceiling, repeat while alternating across several different `/api/ai/*` routes — you should be cut off at 30 requests in the minute even though no single route reached 12.

5.8 SEO, PWA and images

- ❑ Open `/robots.txt` — expect the disallow list and a sitemap line.
- ❑ Open `/sitemap.xml` — expect four public URLs and no dashboard routes.
- ❑ Open `/manifest.webmanifest` — expect the BrandBridge PWA metadata.
- ❑ In DevTools → Application → Manifest, confirm the manifest parses with no errors.
- ❑ On a long list page such as the freelancer marketplace, open DevTools → Network → Img, then scroll. Images should load progressively as they enter the viewport rather than all at once on page load.

6 File index

Created — 14 files

Path	Purpose
src/app/error.tsx	Root error boundary
src/app/global-error.tsx	Root layout error boundary
src/app/not-found.tsx	Branded 404
src/app/loading.tsx	Root loading state
src/app/(protected)/loading.tsx	Dashboard skeleton
src/app/(protected)/error.tsx	In-shell dashboard error panel
src/app/sitemap.ts	Public route sitemap
src/app/robots.ts	Crawler rules
src/app/manifest.ts	PWA manifest
src/lib/rate-limit.ts	Rate limiter
src/lib/rate-limit.test.ts	Rate limiter tests
src/lib/profile-completeness.test.ts	Completeness scoring tests
src/lib/roles.test.ts	Role and navigation tests
vitest.config.ts	Test runner configuration

Modified — 48 files

Area	Files
Critical bug fix	BrandProfileForm.tsx
Messaging	ChatWindow.tsx · MessagesCenter.tsx
AI rate limiting	api/ai/{campaign-brief, campaign-content-ideas, external-brands, hub, match, messages, outreach, product-match, proposal}/route.ts
App shell	app/layout.tsx · styles/brandbridge.css · auth/AuthShell.tsx · DashboardHeader.tsx
Modals	CampaignDetailModal · CampaignFormModal · CollaborationDetailModal · FreelancerDetailModal · HireFormModal · HireDeliverablesModal · GlobalSearchModal · BrandDetailModal · ProposalPreviewModal · TasksModal · DocumentsModal · NotificationCenterModal · DashboardCustomizer
Grids and layout	ProfileForm.tsx · ai-hub/page.tsx · products/[id]/page.tsx · hirer/ProjectsPage.tsx

Area	Files
Lazy images	18 files carrying raw tags
Tooling	package.json · package-lock.json

Dependencies added (dev only): vitest, @vitejs/plugin-react, jsdom, @testing-library/react, @testing-library/jest-dom. No runtime dependency was added.

7 Limits and recommended next steps

What was deliberately not done

Item	Reason
Full <code>next/image</code> migration	Uploads take two different shapes. When Cloudinary credentials are set, images are Cloudinary URLs, which <code>next/image</code> handles and which are already allow-listed in <code>next.config.ts</code> . When those credentials are absent the upload route falls back to inline base64 <code>data:</code> URLs, which <code>next/image</code> cannot optimise and would need <code>unoptimized</code> to render at all. Migrating blindly risked breaking image display in exactly the fallback path. Lazy loading was applied instead, which is safe for both shapes.
Real-time transport (SSE or WebSocket)	Polling was chosen because it required no infrastructure change and closed the functional gap immediately. It is the right first step, not the final answer — see below.
Distributed rate limiting	Would require Redis or Upstash, which is an infrastructure decision that is yours to make. The code is structured so the swap is a single function.
Component and end-to-end tests	The 58 tests cover pure logic. Component tests are wired and ready — <code>@testing-library/react</code> and <code>jsdom</code> are installed and the config supports a per-file jsdom docblock — but no component suites were written in this pass.

Recommended next, in order

1. **Replace chat polling with Server-Sent Events.** At 5-second polling, 100 concurrent users generate roughly 72,000 requests an hour against MongoDB. SSE reduces that to one open connection each and delivers messages instantly.
2. **Move rate-limit counters to Redis** before any multi-instance deployment, otherwise the ceiling scales with your instance count.
3. **Add response caching to the AI routes.** Rate limiting caps abuse but identical prompts still cost a full Gemini call every time.
4. **Write component tests for the brand profile form.** It is the most complex stateful form in the app and it is where this pass found a bug that no type check could catch.
5. **Verify on real iOS hardware.** The zoom-on-focus fix in 3.1 cannot be reproduced in desktop device emulation.

Repository state

Every change sits uncommitted in the working tree on `main`, as requested. Nothing has been staged, committed or pushed, and the dev server was never started. Run `git diff` to review, or `git checkout -- .` to discard everything.